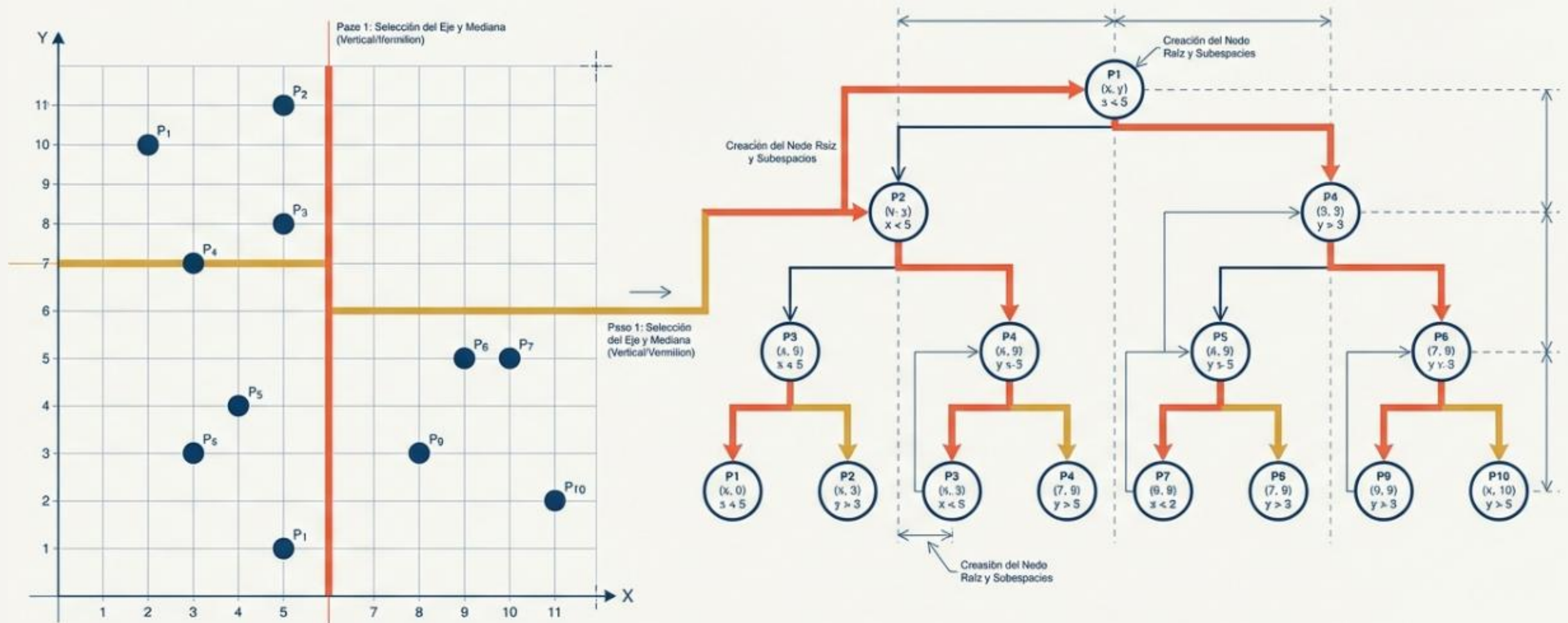


Árboles multidimensionales KDTree

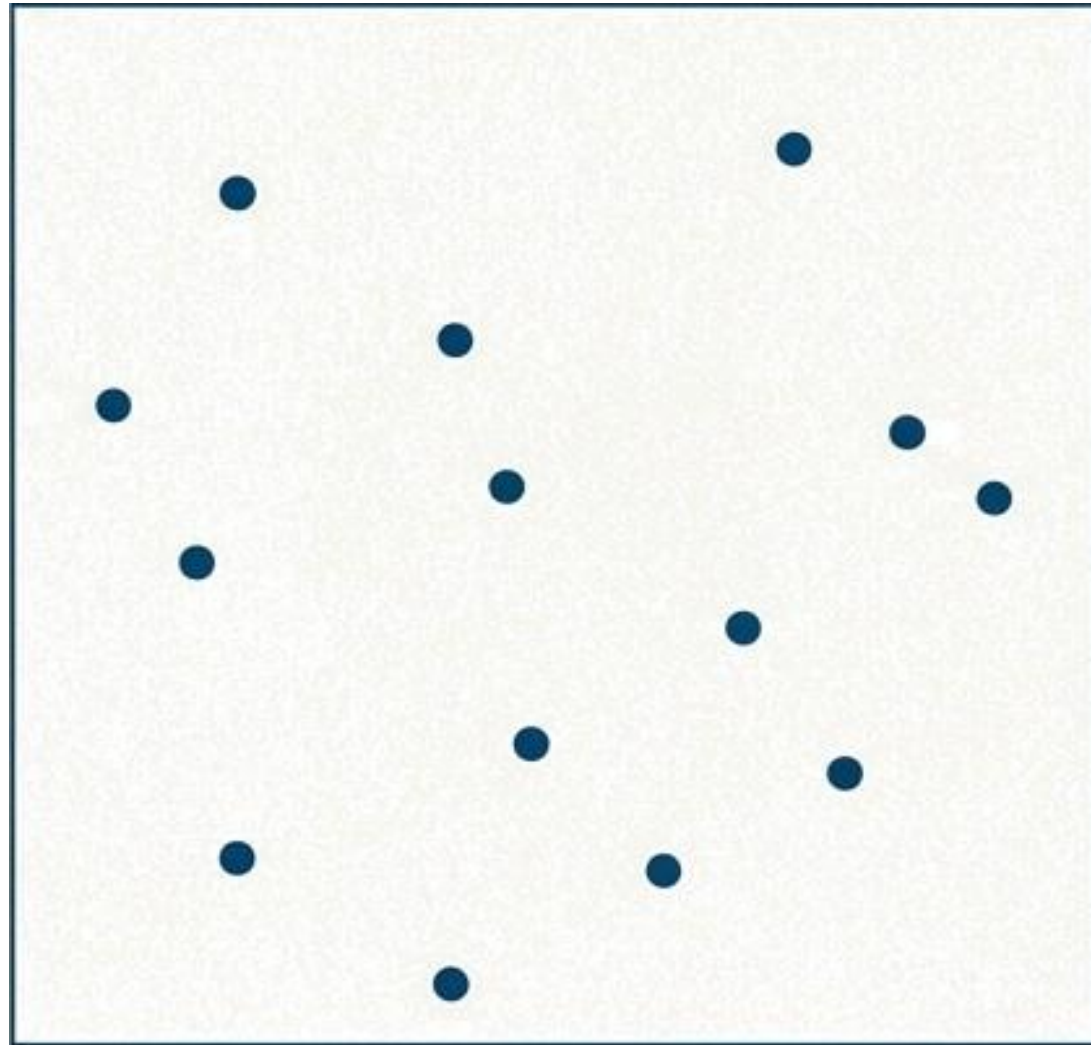
La partición espacial paso a paso



¿Qué es un Árbol k-d?

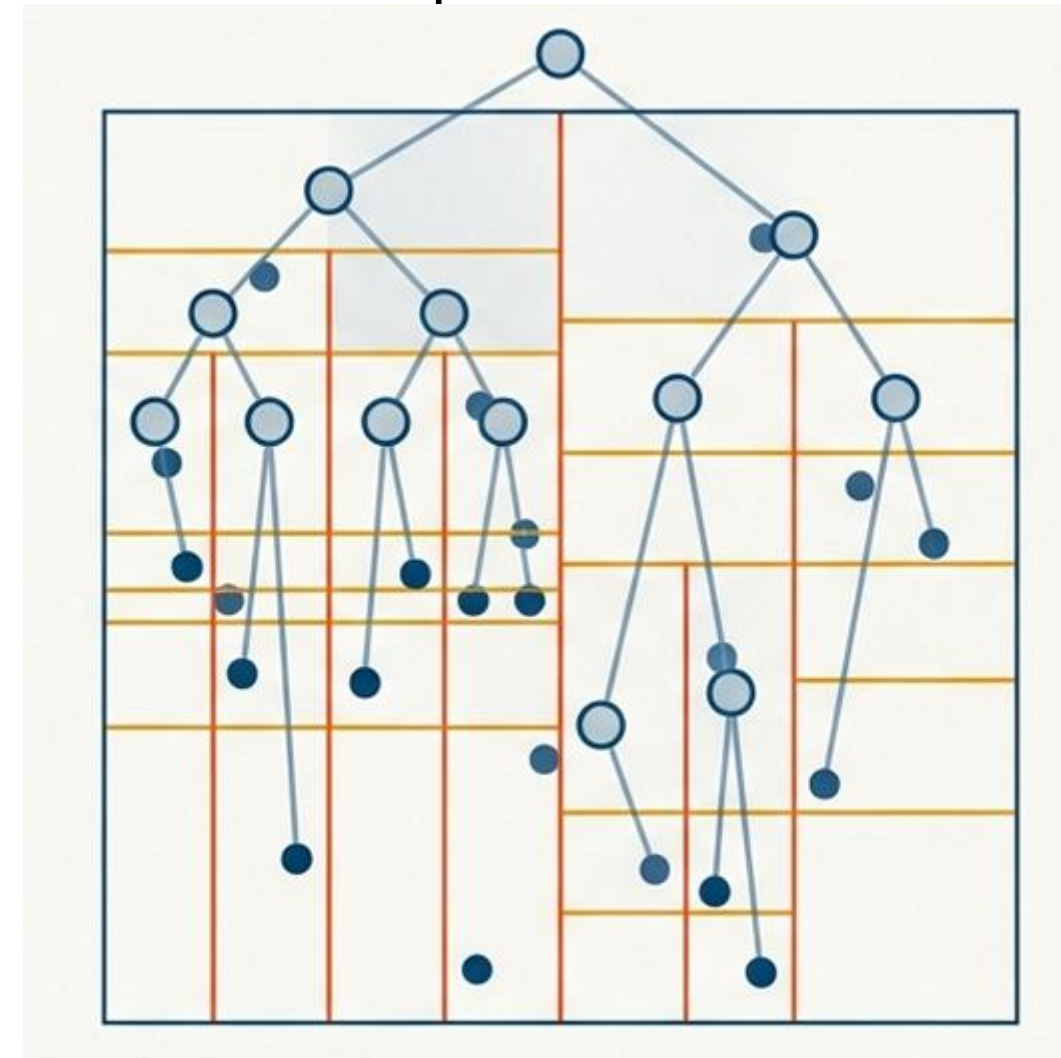
El problema:

Dado un conjunto grande de puntos, encontrar el (o los) vecinos más cercanos a un punto consulta sin tener que comparar contra todos (lo cual cuesta $O(n)$).

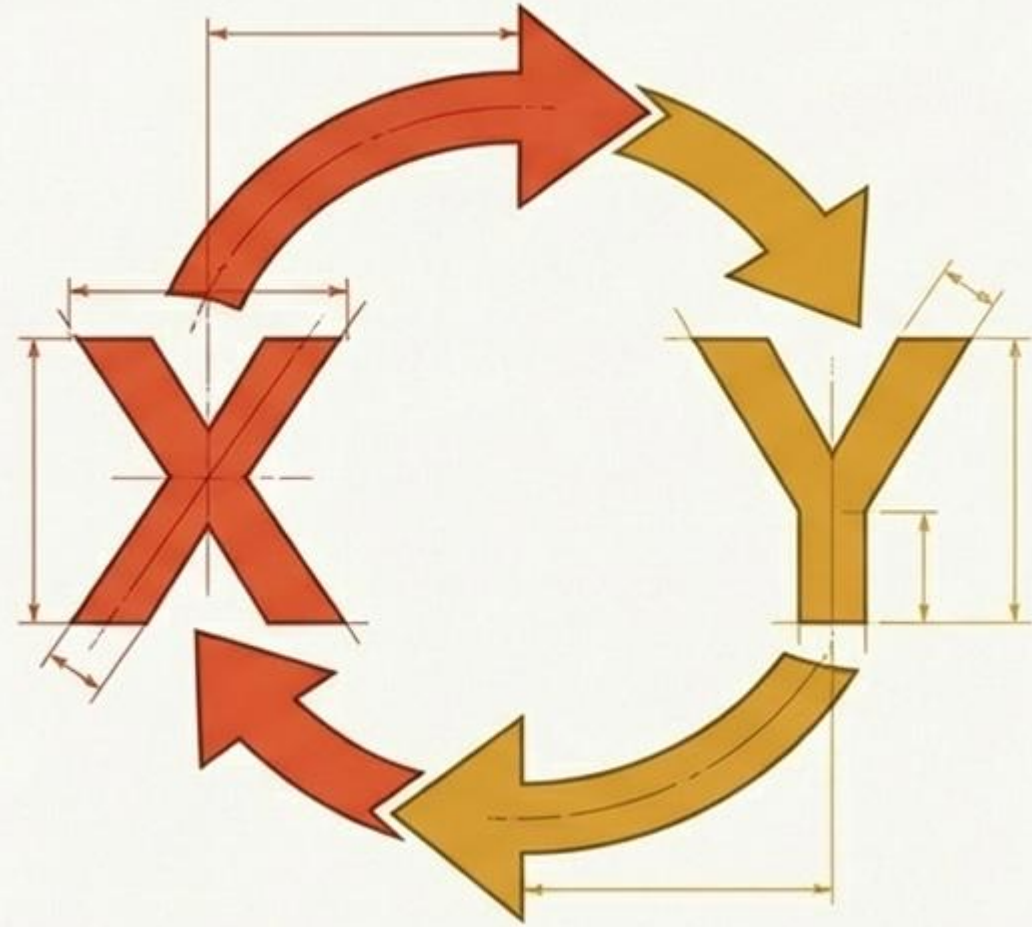


La solución:

Organizar los puntos en un árbol que divide el espacio, de forma que durante la búsqueda se puedan **descartar regiones enteras** y solo explorar las relevantes, reduciendo mucho el número de comparaciones.

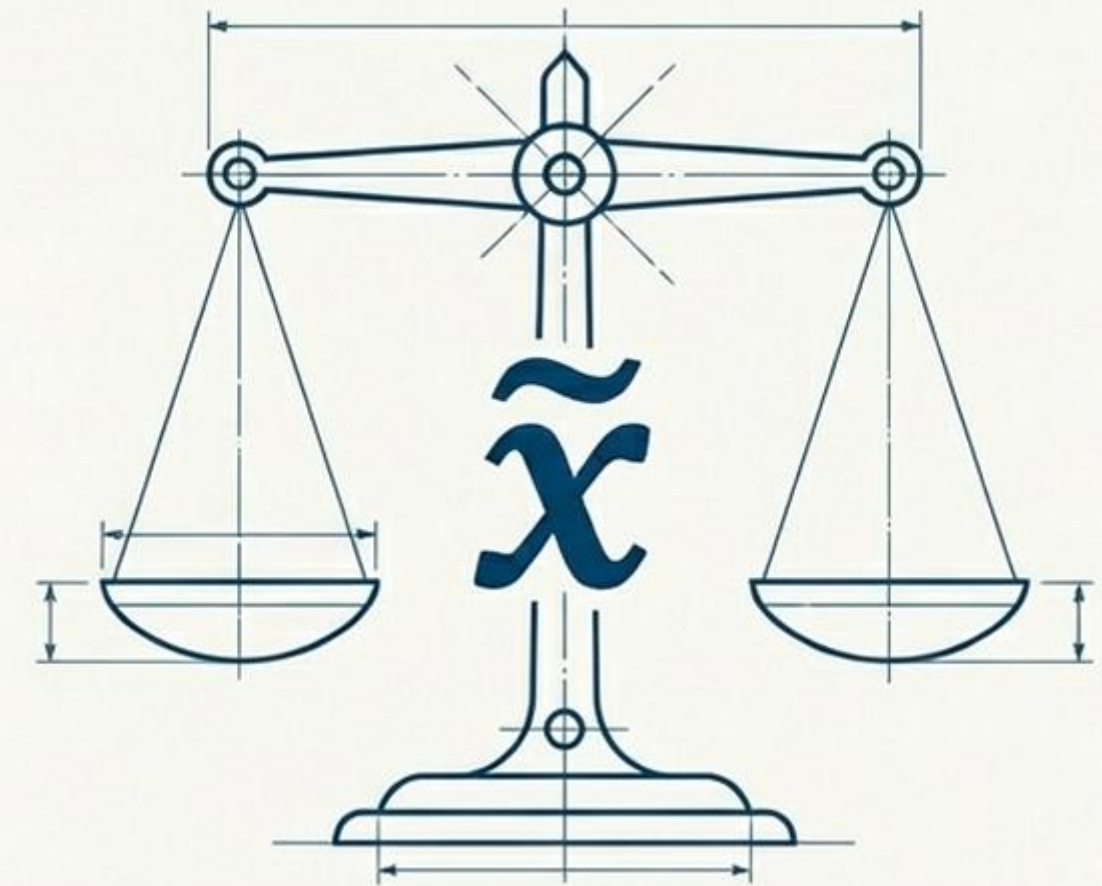


Las Dos Reglas de Oro



Regla 1: Alternancia de Ejes.

Cortamos el espacio verticalmente en el eje X, luego horizontalmente en el eje Y, y repetimos el ciclo en cada nivel de profundidad.



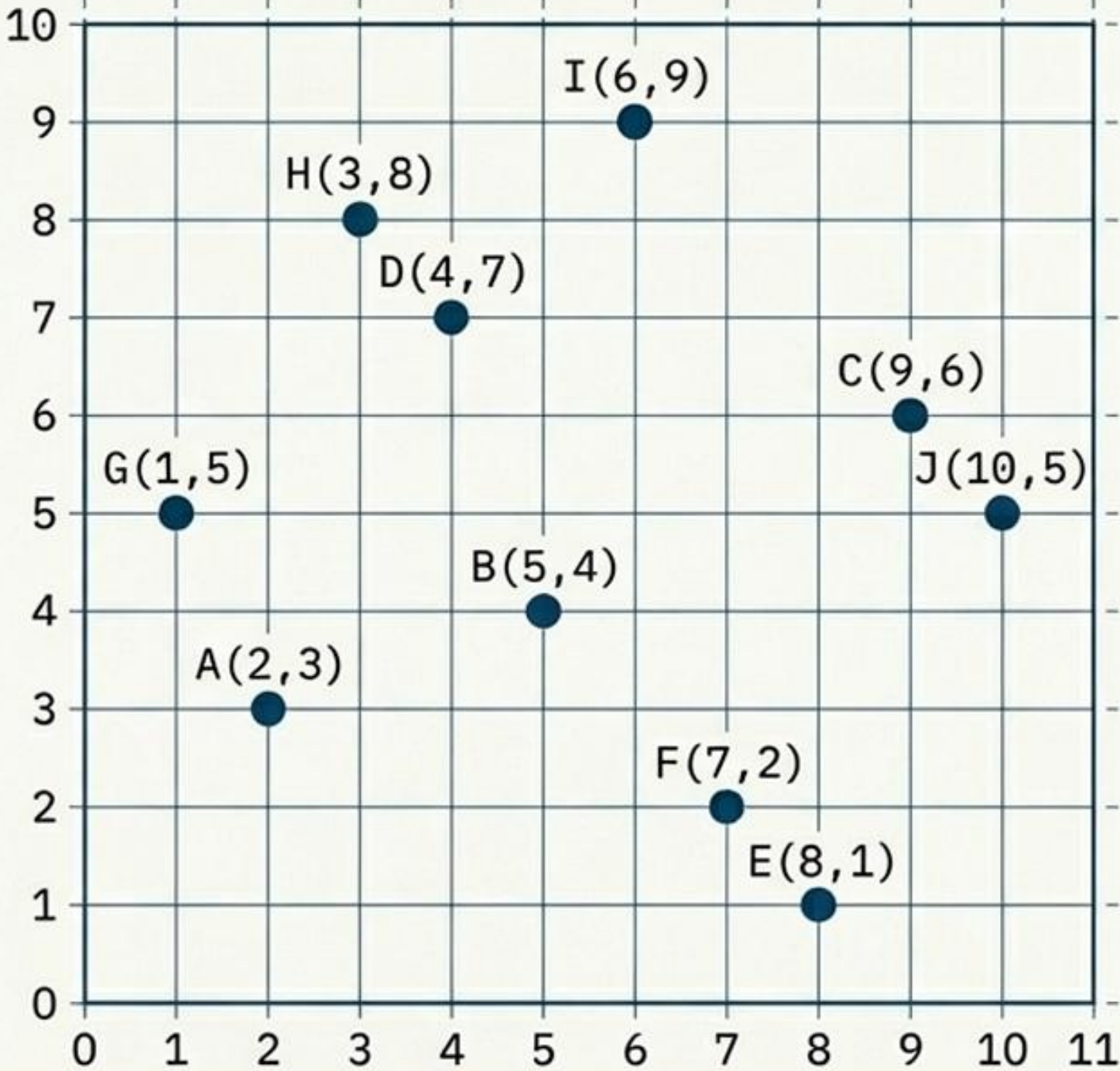
Regla 2: La Mediana es el Pivote.

Seleccionar el punto medio exacto garantiza que la rritad de los datos vaya a cada subárbol, manteniendo el equilibrio jerárquico perfecto.

Nuestro Universo de Datos

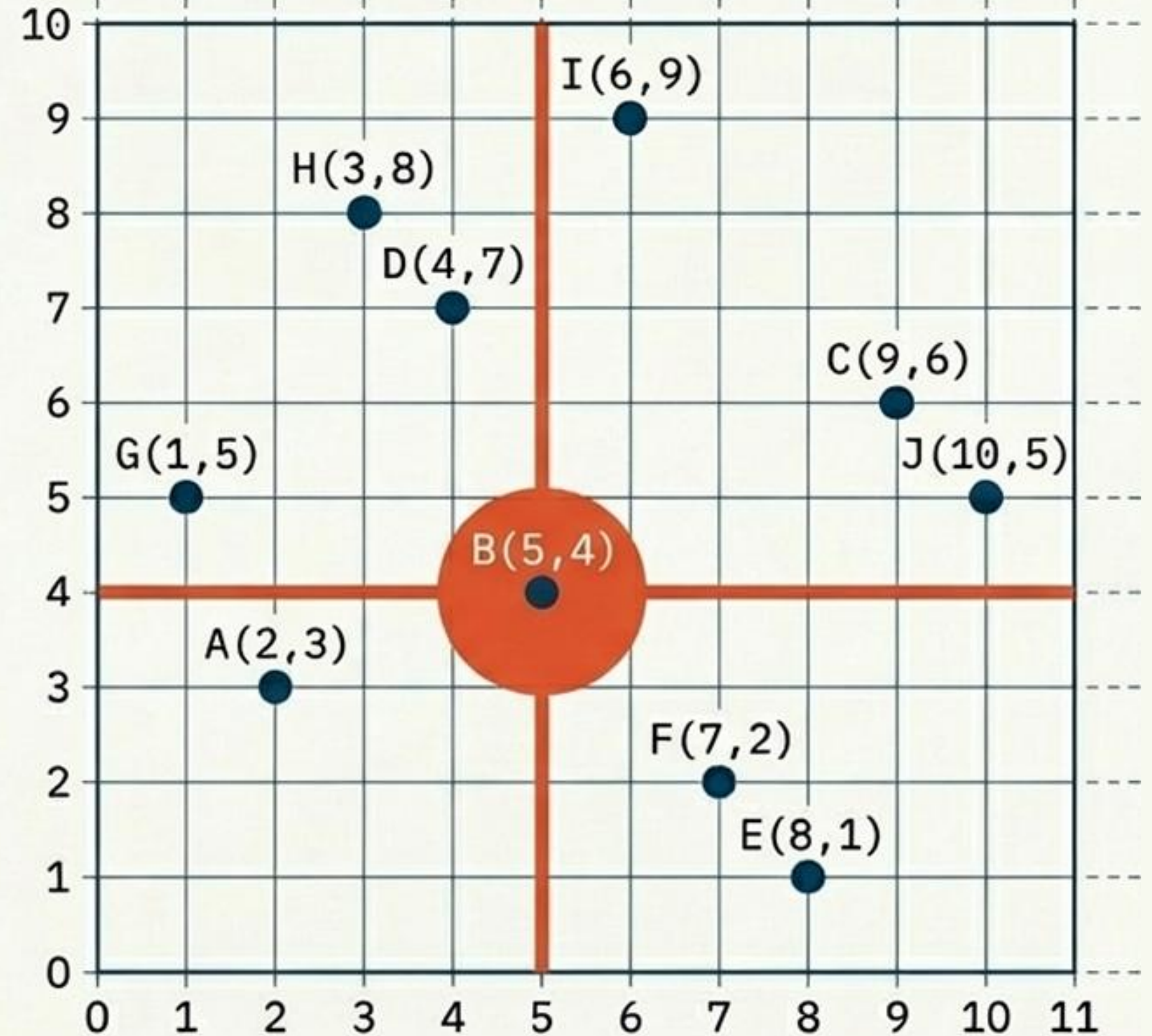
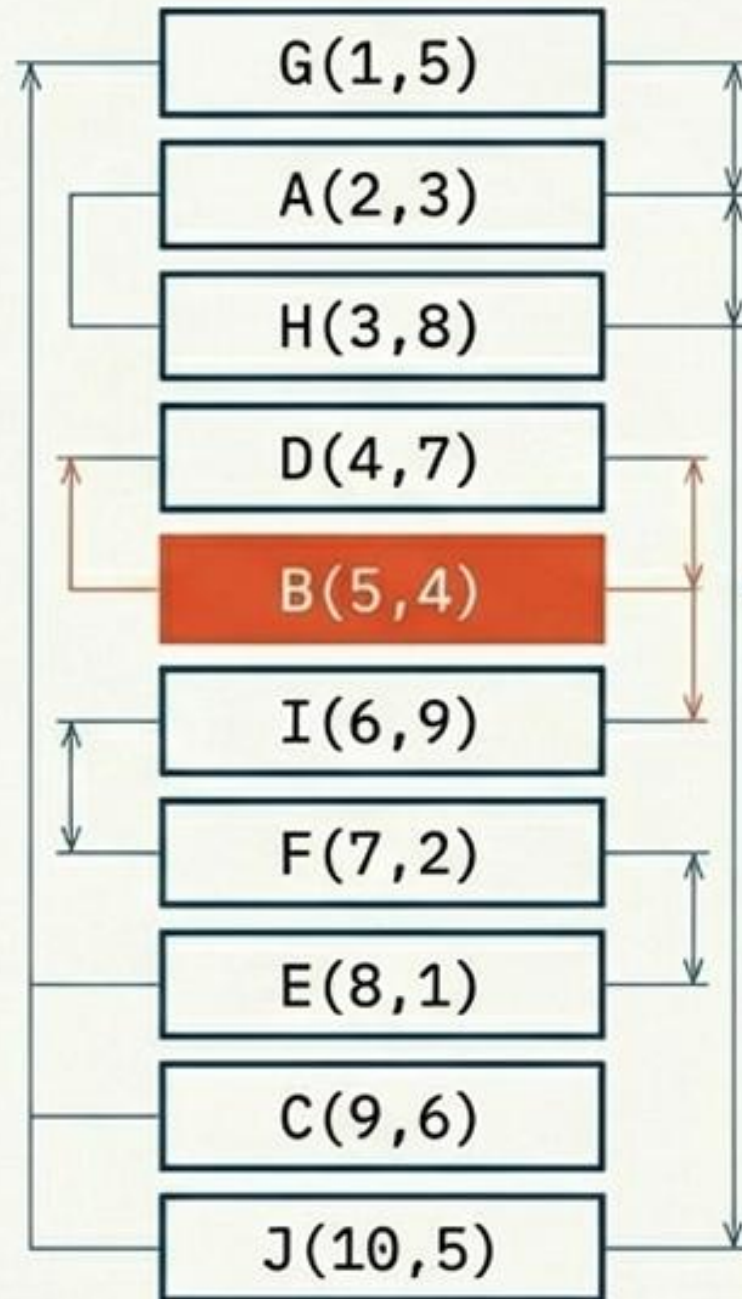
El desafío: Organizar 10 vectores bidimensionales (k=2) en una estructura de búsqueda optimizada

A(2,3)		B(5,4)		C(9,6)		D(4,7)		E(8,1)
F(7,2)		G(1,5)		H(3,8)		I(6,9)		J(10,5)



Nivel 0: El Primer Pivote (Eje X)

1. **Dimensión:** Eje X (Profundidad 0).
2. **Ordenamiento:** Alineamos todos los puntos por su valor en X.
3. **Pivote:** Seleccionamos la mediana aritmética.

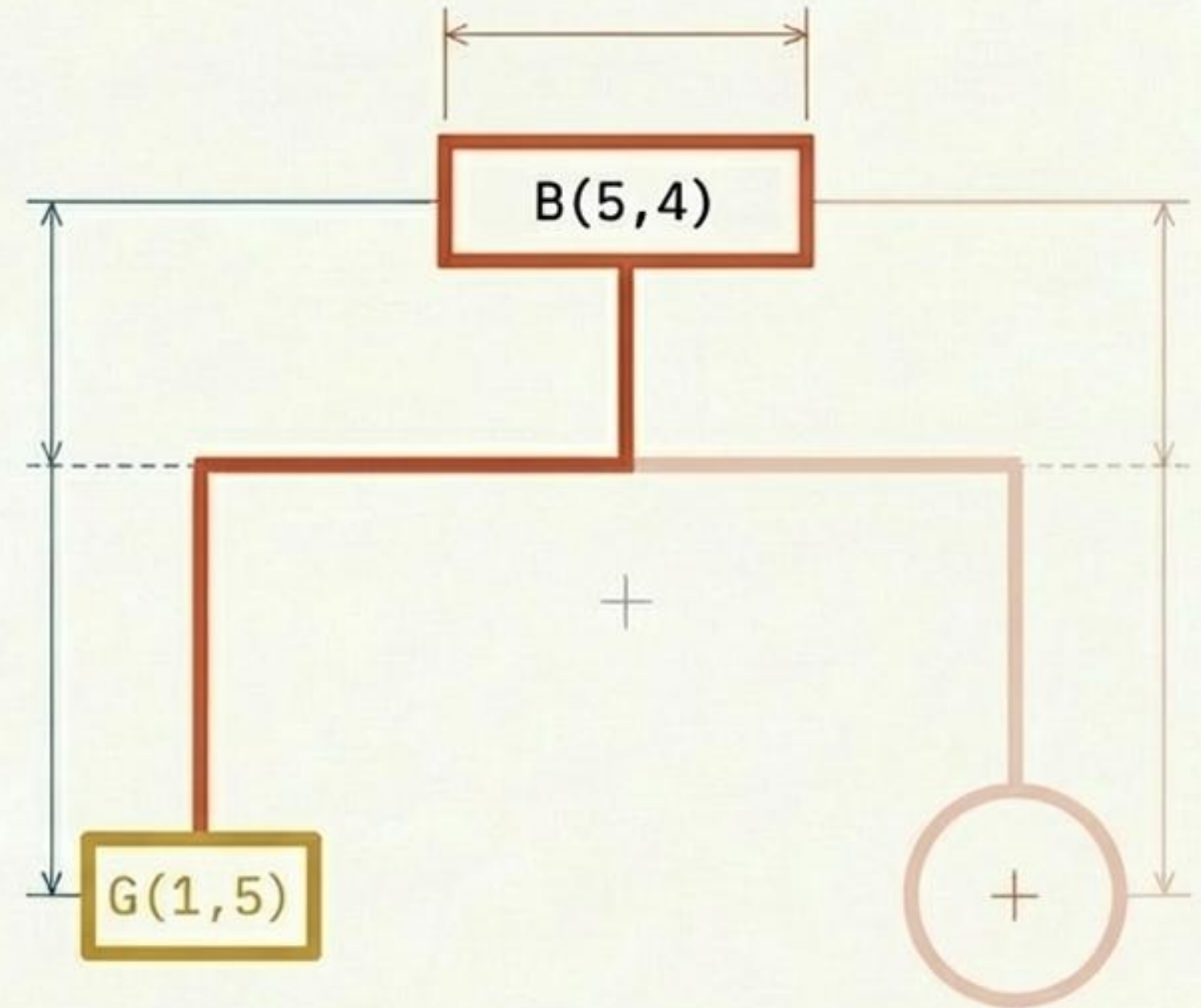
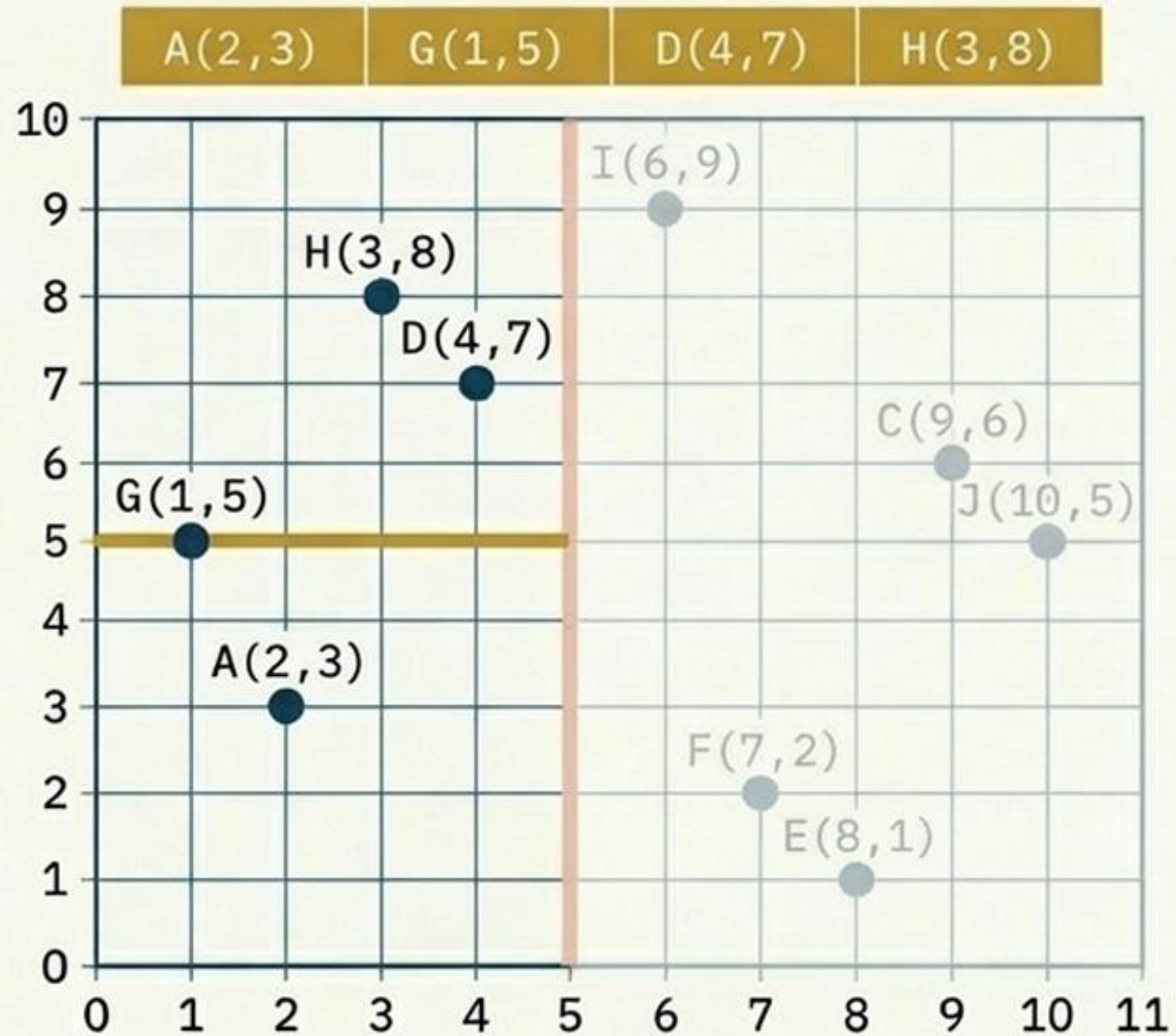


Nivel 1(Izquierda): Alternando al eje Y

Analizamos el subespacio izquierdo ($X < 5$)

Alternamos al eje Y: Ordenamos por Y

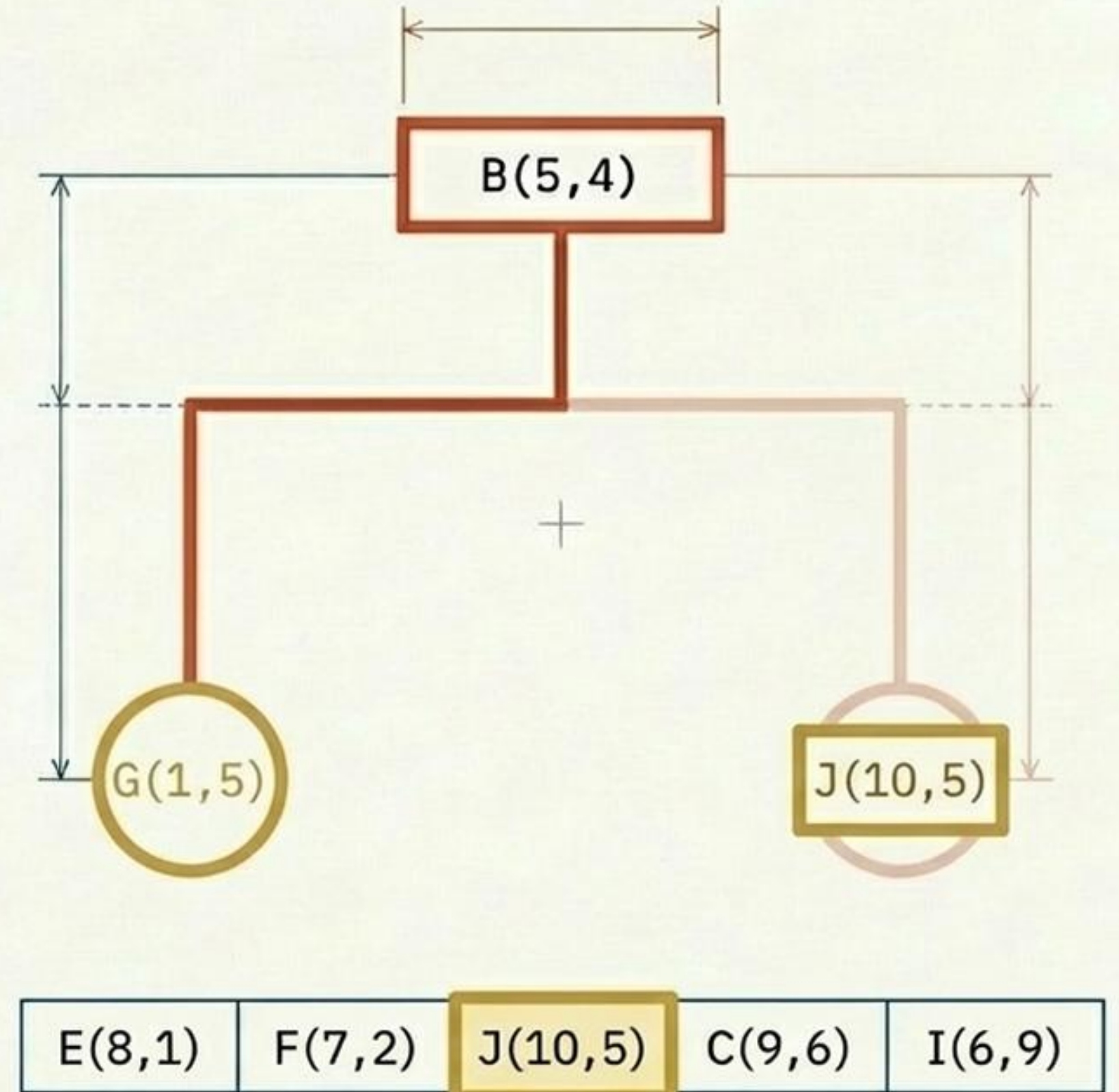
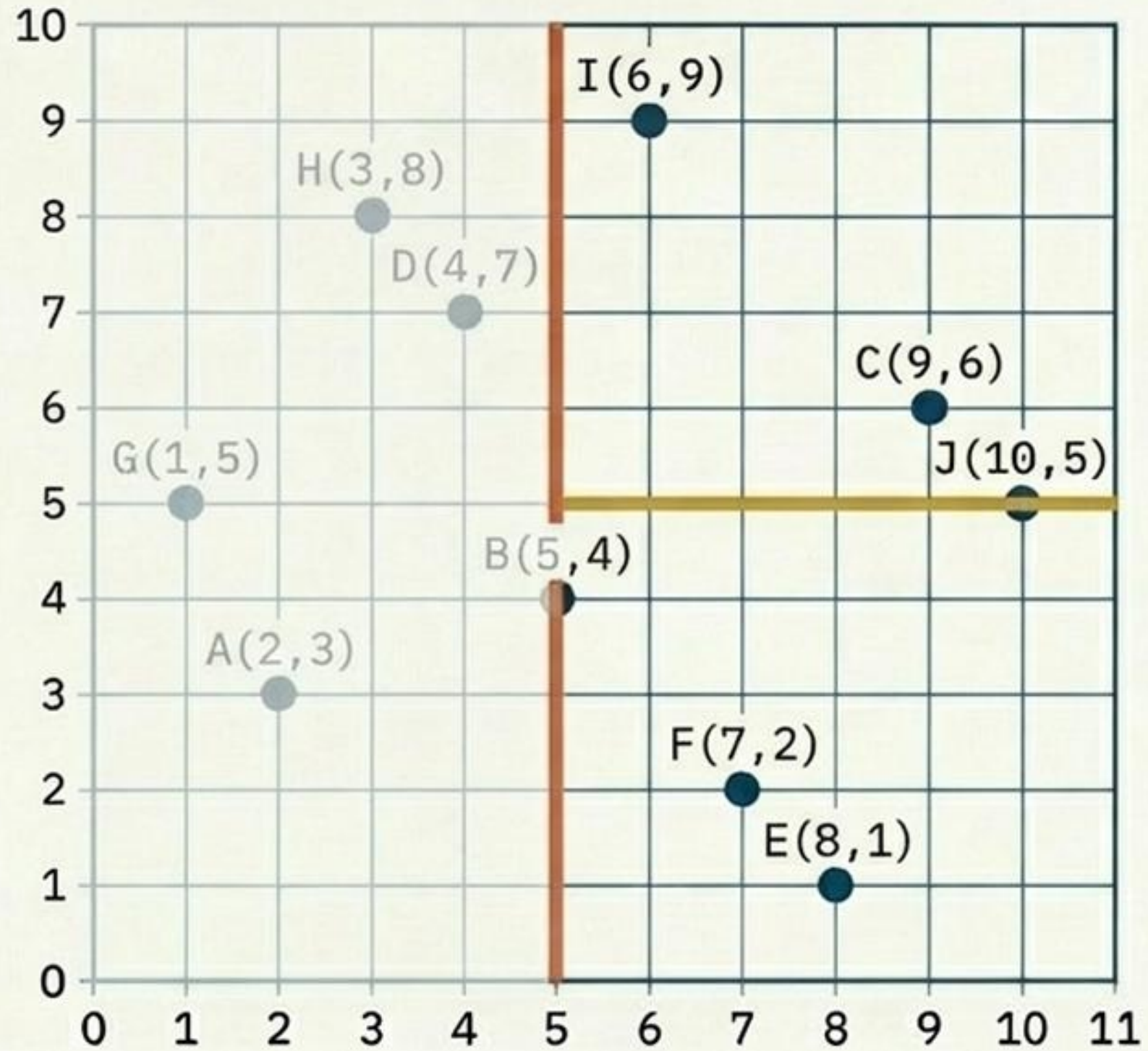
La nueva mediana es $G(1,5)$



Nivel 1(Derecha): Completa el Nivel 1

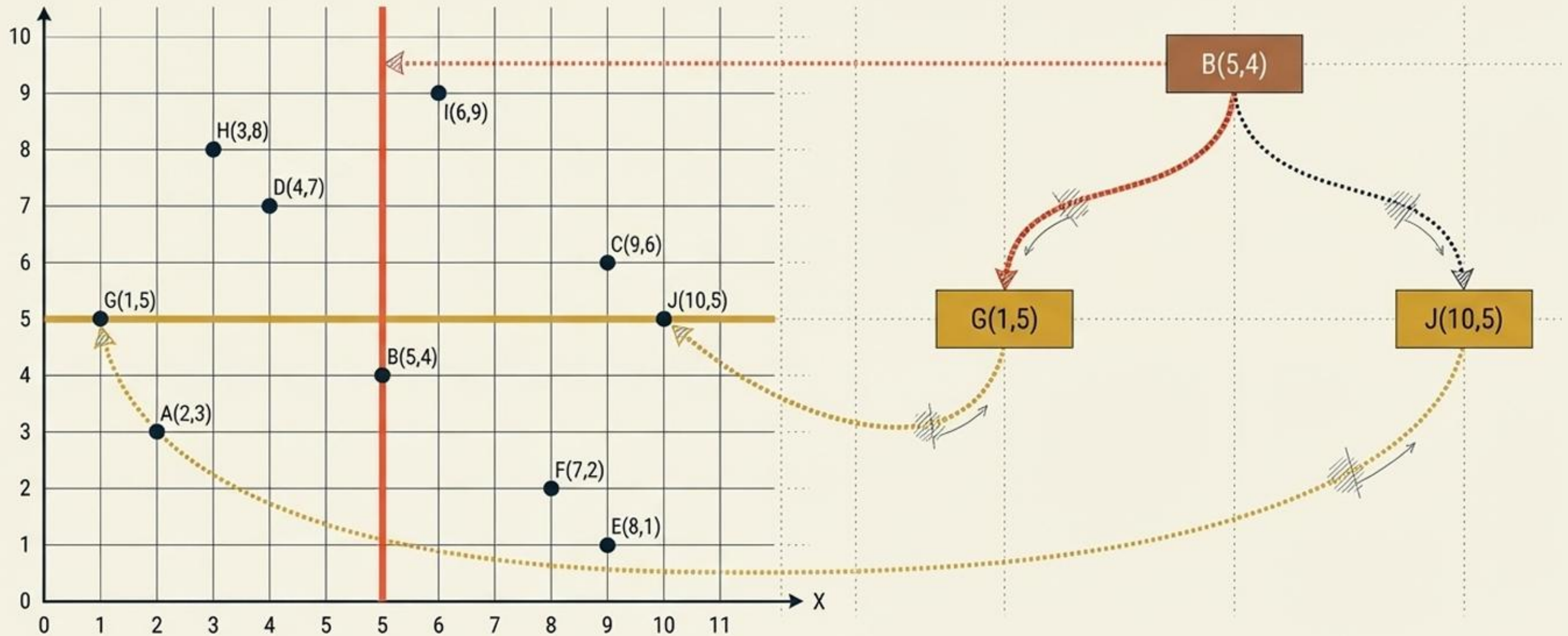
Analizamos el subespacio izquierdo ($X \geq 5$)

Ordenamos por Y, La mediana es G(10,5)



El espacio toma forma

Observa la simetría. El uso estricto de la mediana asegura que, aunque los rectángulos geométricos varíen en tamaño físico, el balance poblacional de los datos es matemáticamente perfecto.

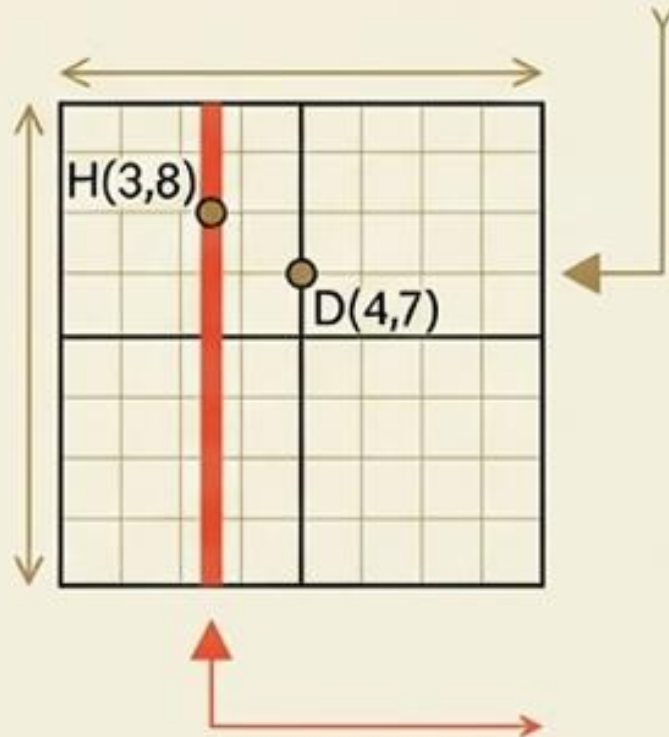


Nivel 2: Retorno al Eje X (Nodos Hoja)

Sub-Izq-Der

Puntos: H(3,8),
D(4,7).

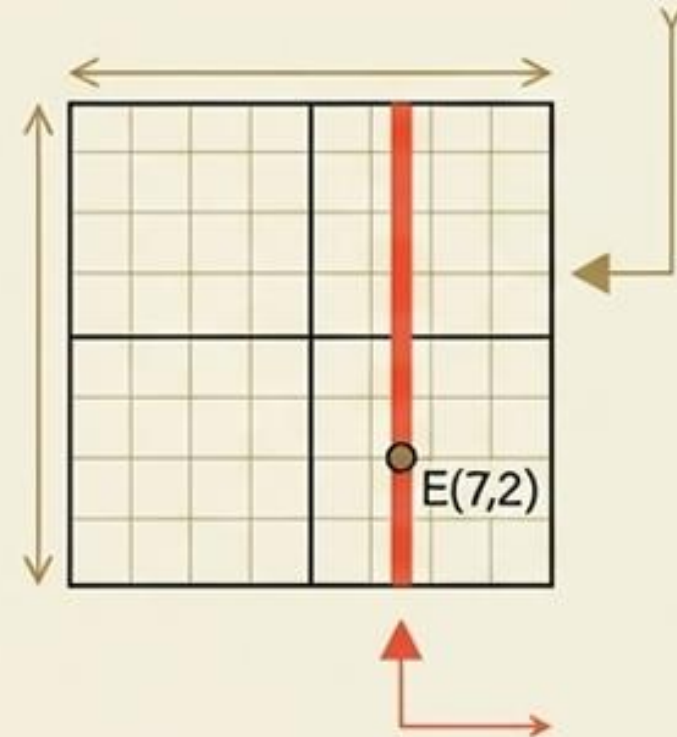
Orden X: H, D.
Mediana: H.



Sub-Der-Izq

Puntos: E(8,1), F(7,2).

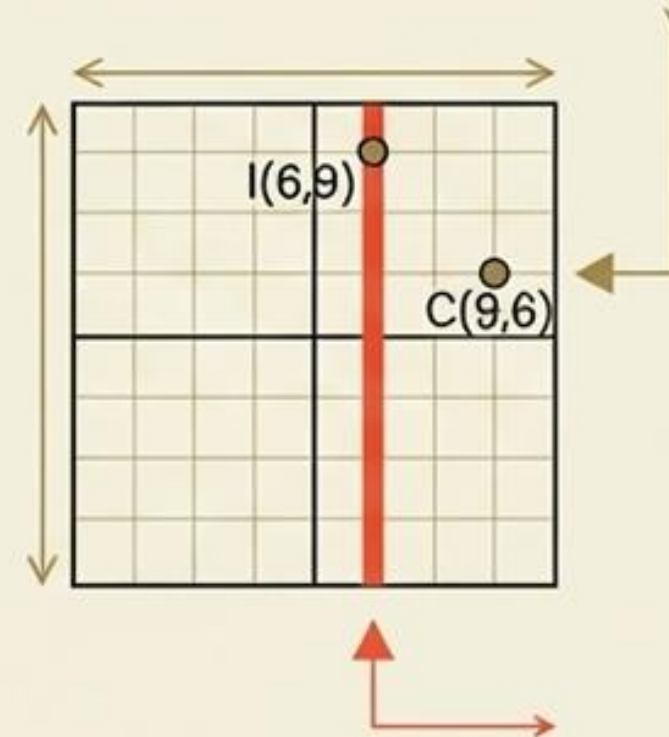
Orden X: F, E.
Mediana: F.



Sub-Der-Der

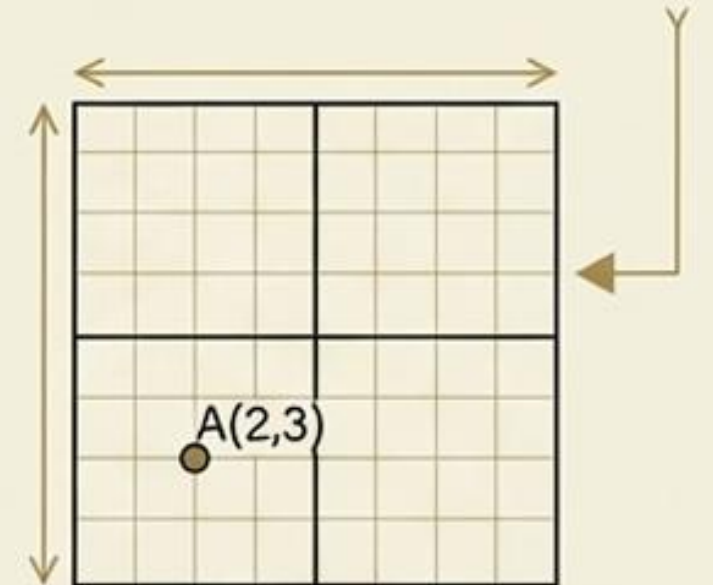
Puntos: I(6,9), C(9,6).

Orden X: I, C.
Mediana: I.



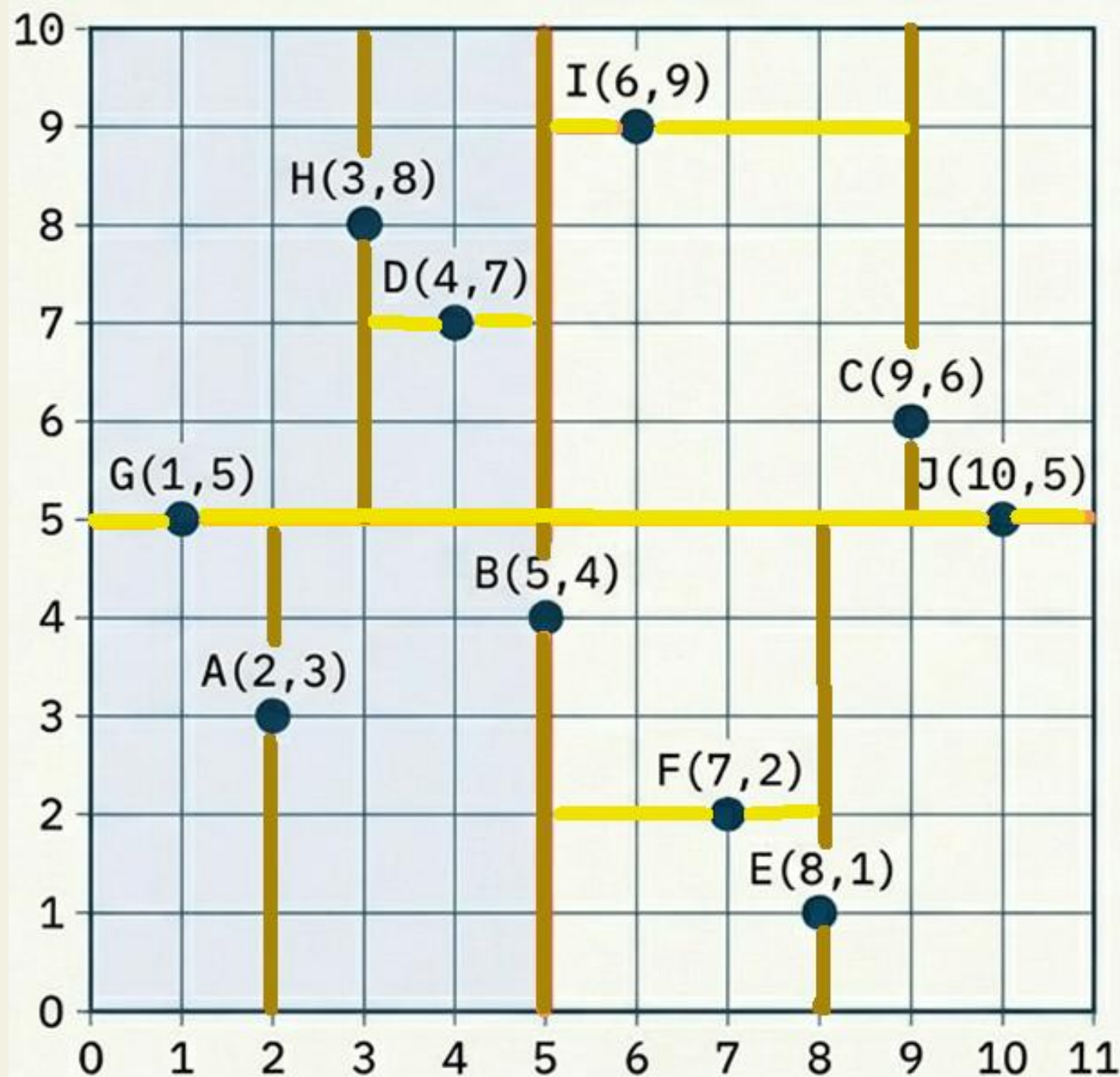
Hojas Solitarias

El punto A(2,3)
queda aislado
matemáticamente
en su propia región
desde el Nivel 1.



Estructura Final

Un espacio continuo indexado jerárquicamente. Cada hoja del árbol es un segmento discreto del universo



G(1,5) A(2,3) H(3,8) D(4,7) **B(5,4)** I(6,9) F(7,2) E(8,1) C(9,6) J(10,5)

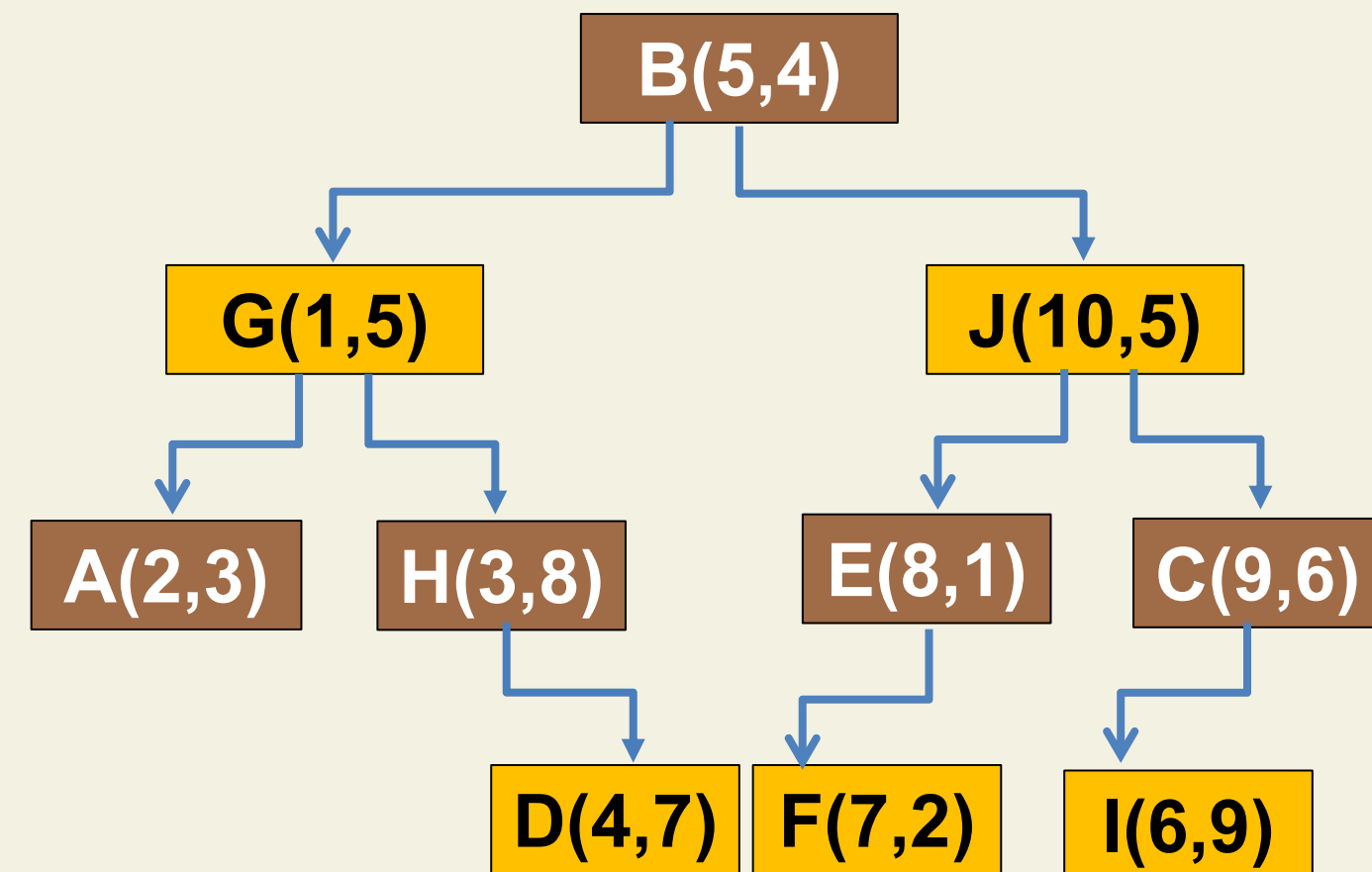
A(2,3) **G(1,5)** D(4,7) H(3,8)

E(8,1) F(7,2) **J(10,5)** C(9,6) I(6,9)

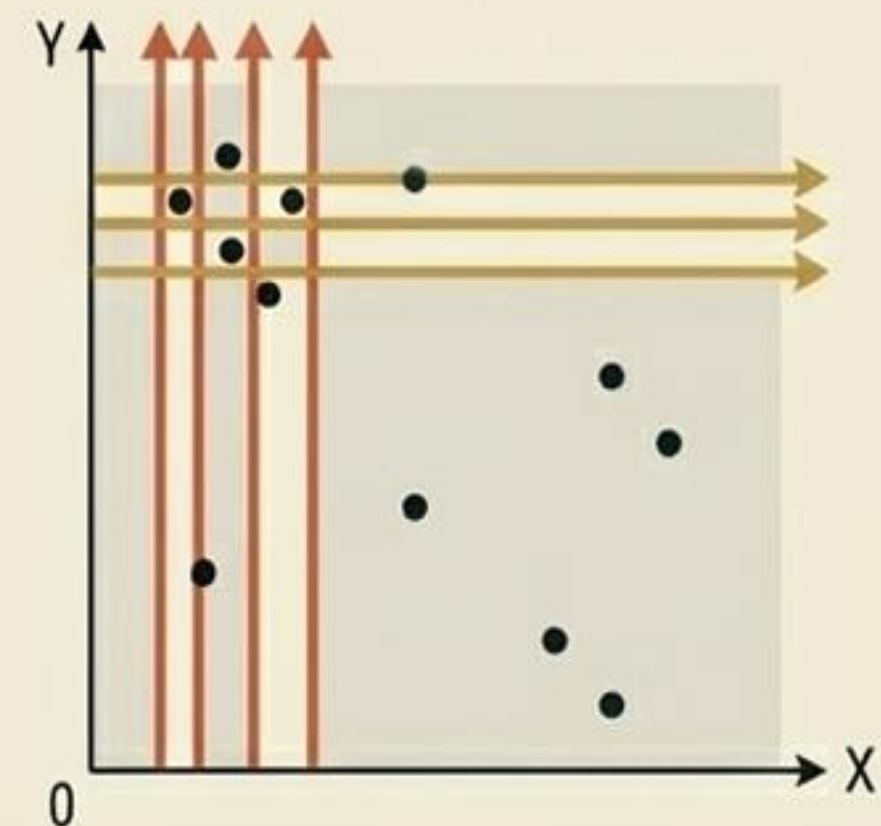
H(3,8) D(4,7)

F(7,2) E(8,1)

I(6,9) C(9,6)

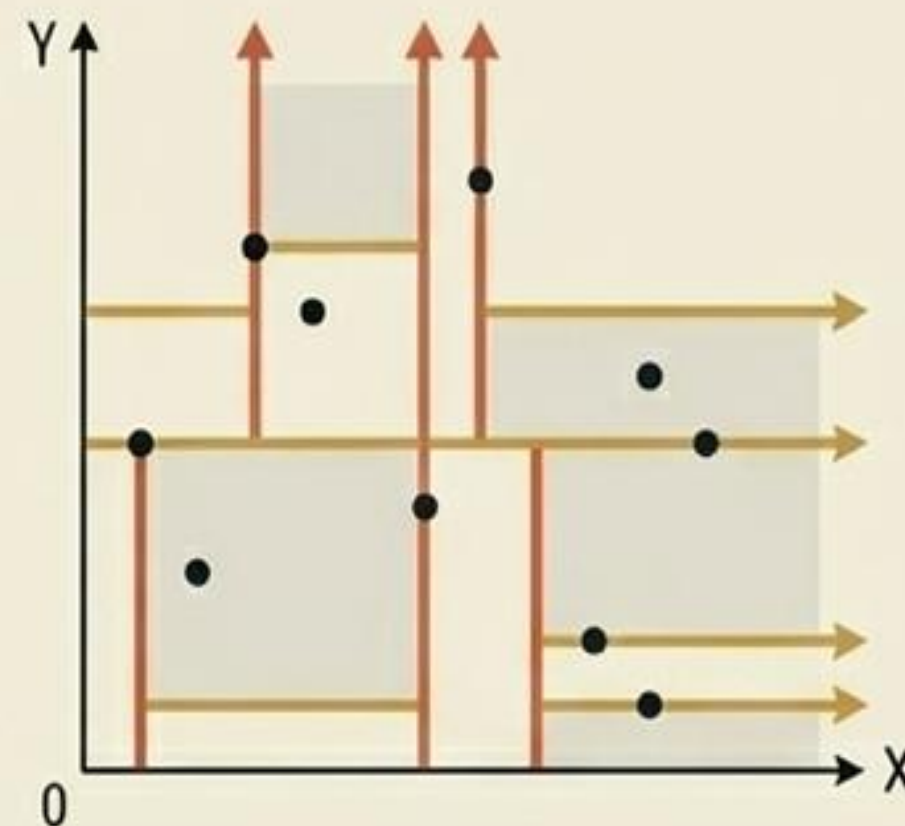
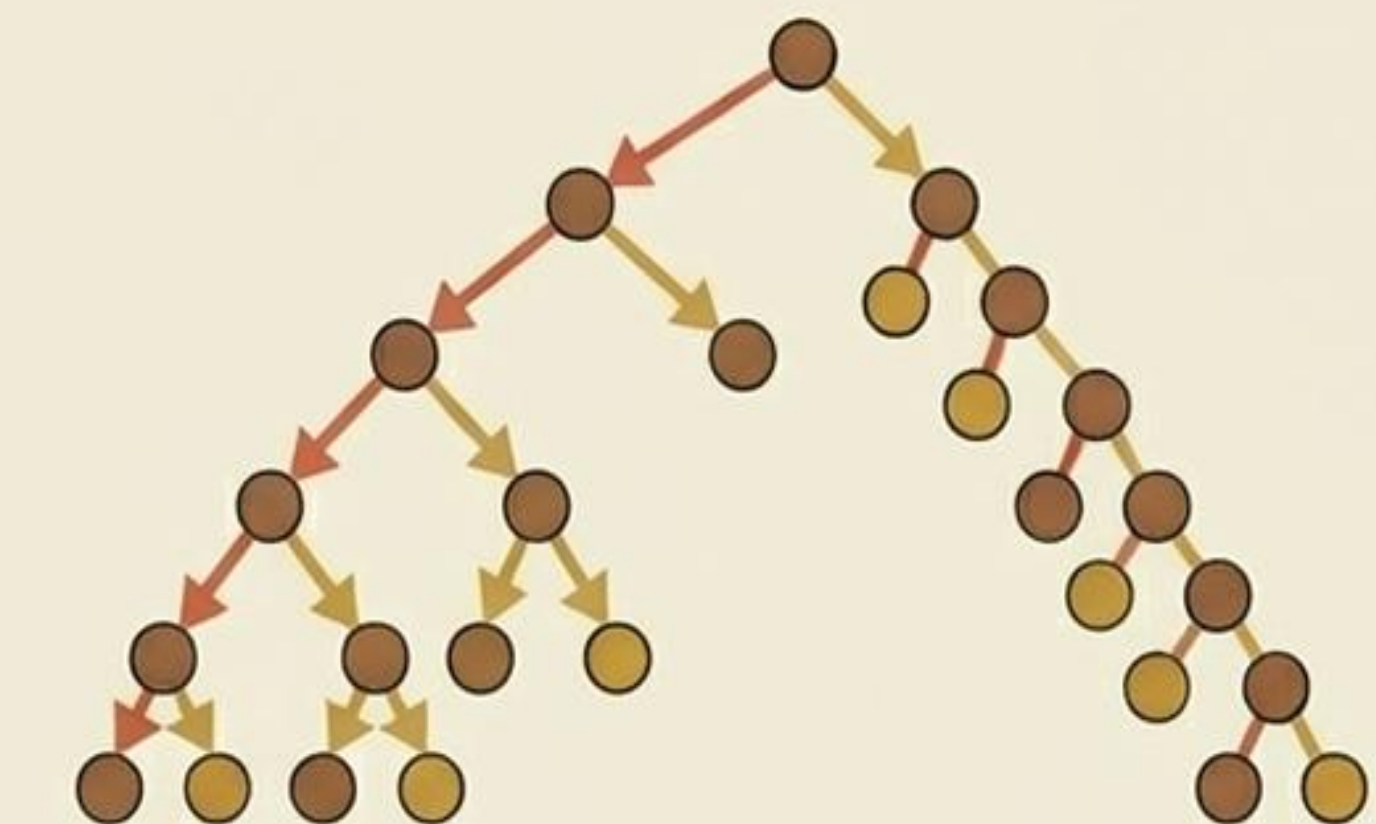


El Peligro del Desequilibrio: ¿Por qué la Mediana?



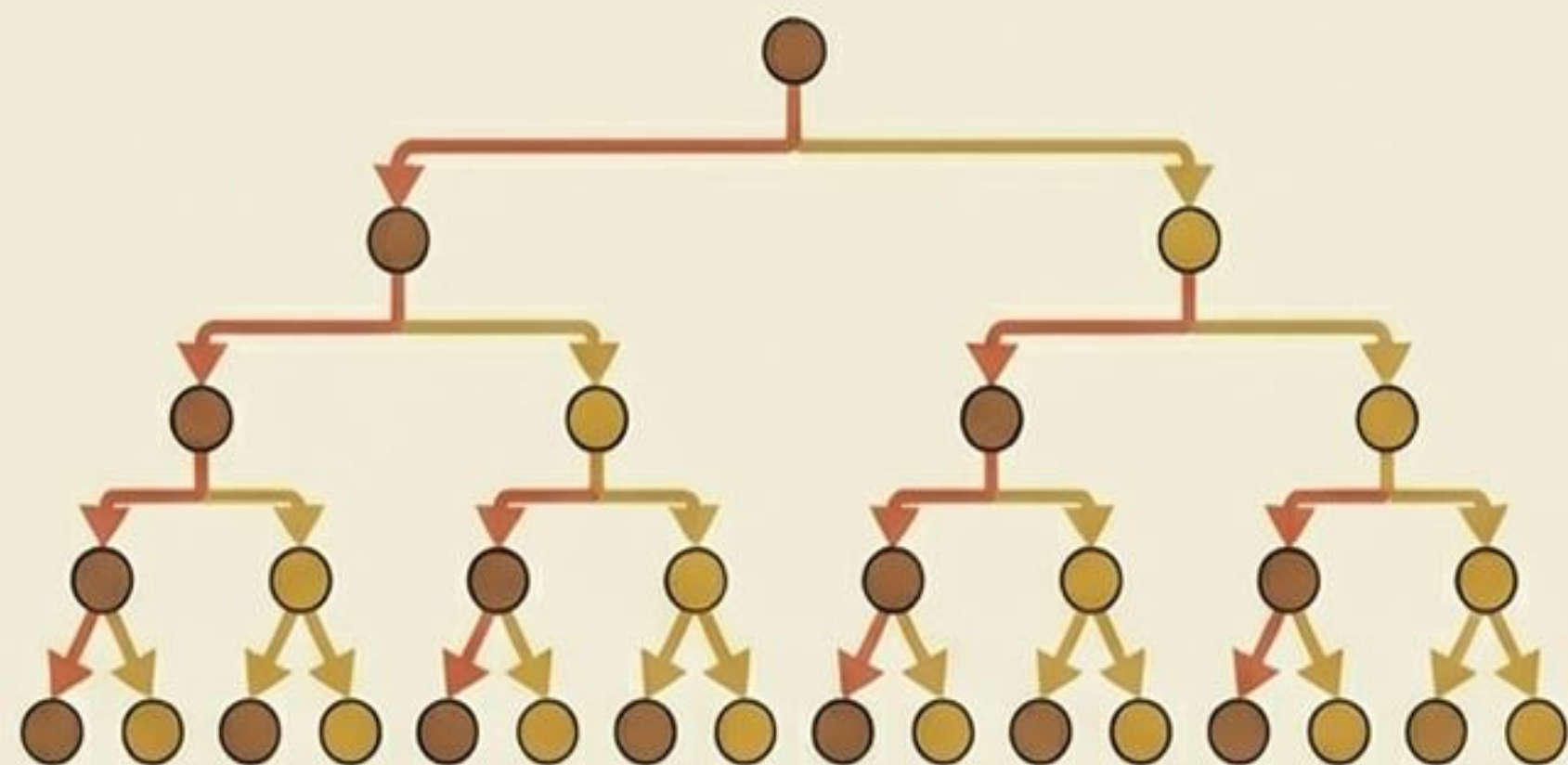
Pivotes Aleatorios

- Tiempo de búsqueda degradado a $O(n)$.



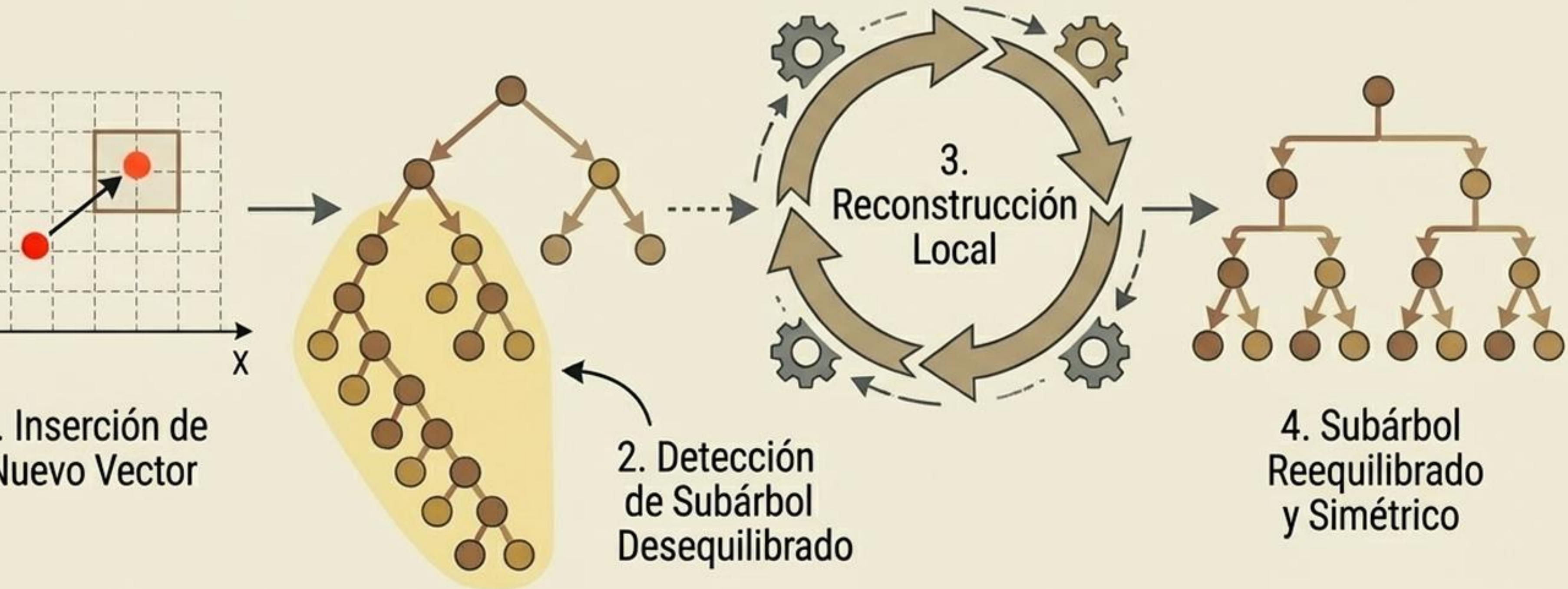
Pivotes Medianos

- Altura del árbol bloqueada en $O(\log n)$. Rendimiento óptimo garantizado.



Árboles Dinámicos y Reequilibrio

En la práctica, los árboles k-d no son estáticos. Al insertar o eliminar vectores, el árbol evalúa su balance. Si una rama se desestabiliza, ese subárbol local se reconstruye sobre la marcha, sin afectar el resto del universo.



El Costo de la Eficiencia (Complejidad)

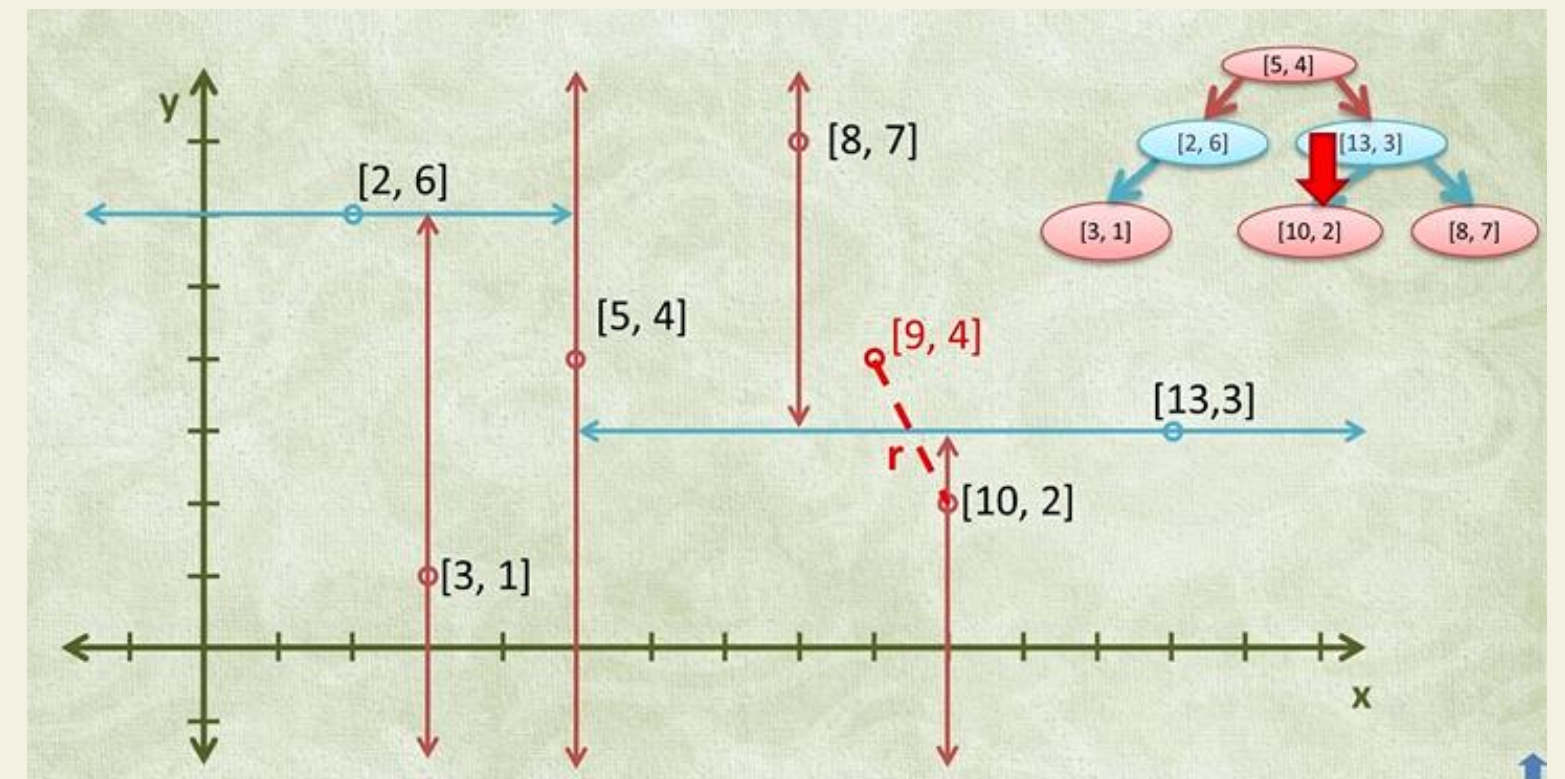
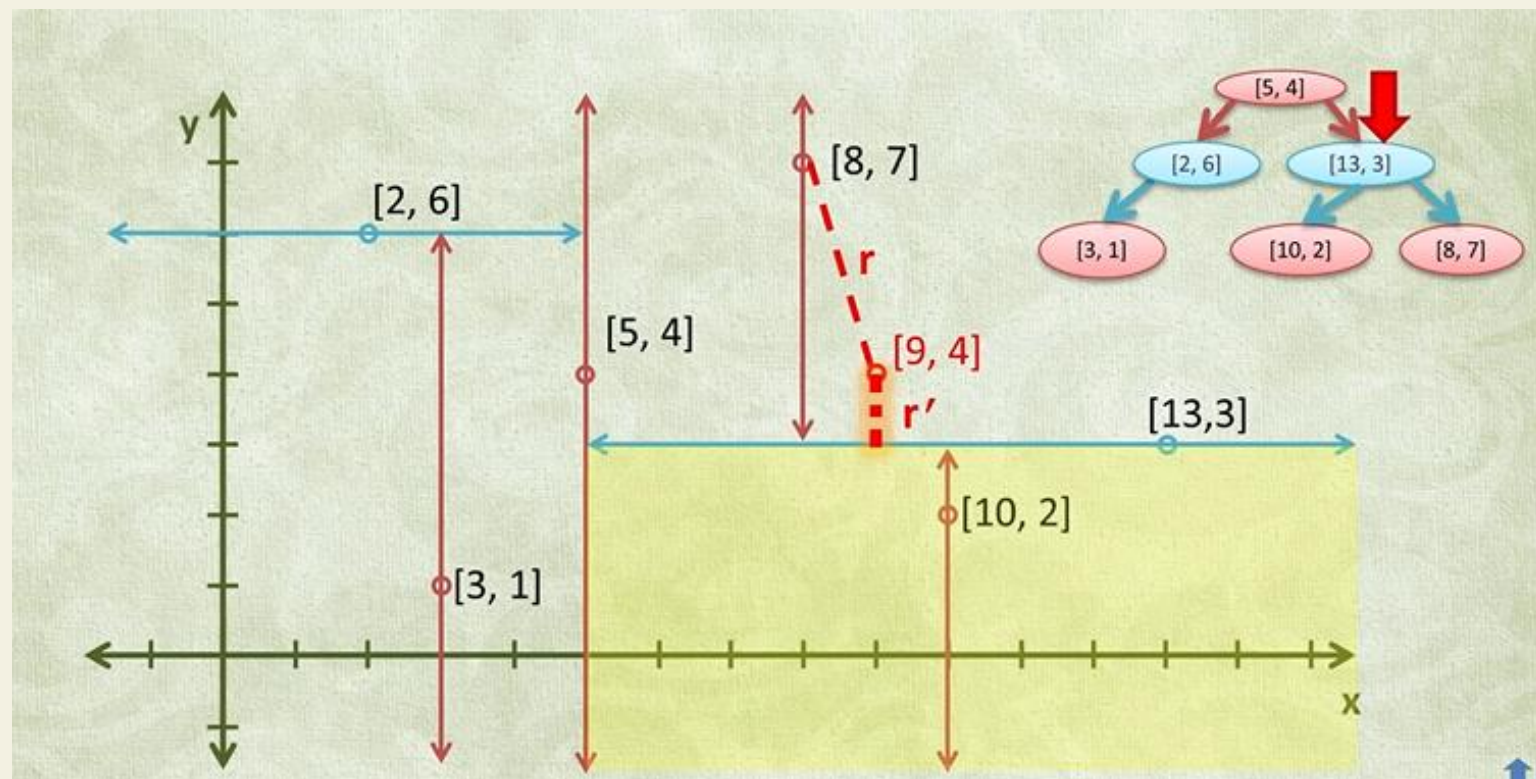
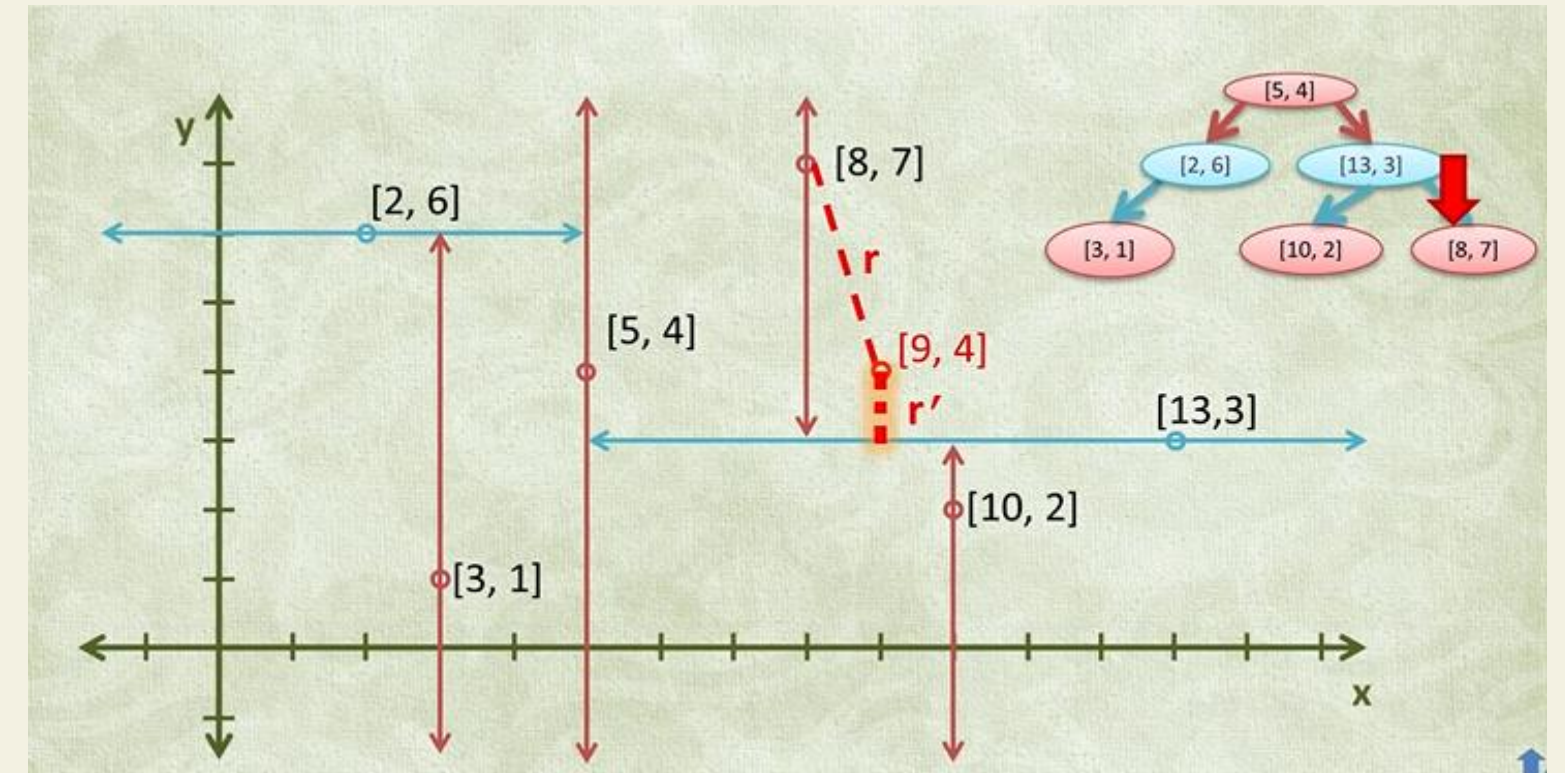
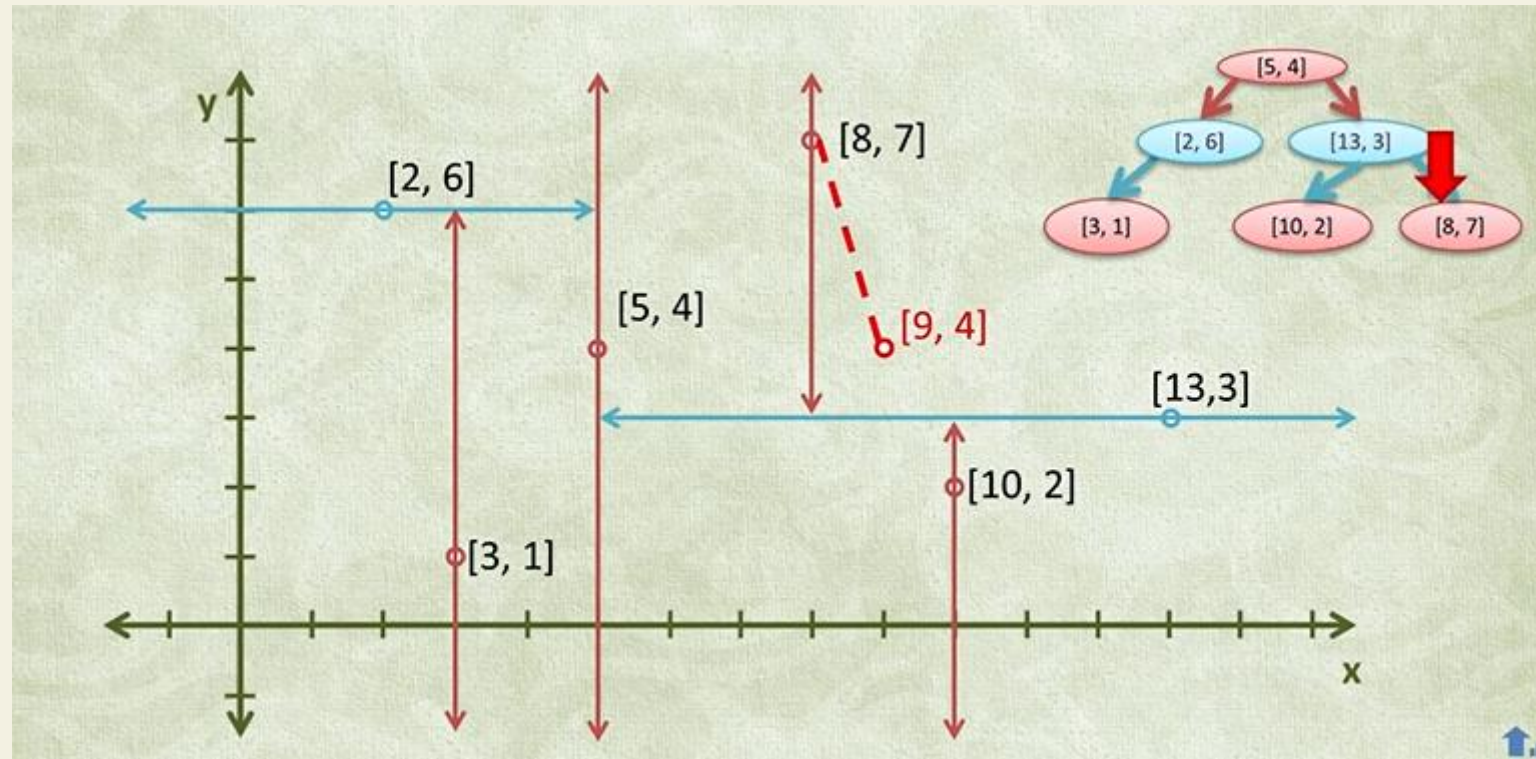


Operación	Tiempo Promedio	Peor Caso
Construcción	$O(n \log n)$	$O(n \log n)$
Búsqueda (Vecino Cercano)	$O(\log n)$	$O(n)$ (si está desbalanceado)
Inserción / Borrado	$O(\log n)$	$O(n)$

Complejidad de Espacio: $O(n)$

Algoritmo KNN (K-Nearest Neighbors)

El algoritmo KNN es un clasificador de aprendizaje supervisado no paramétrico que utiliza la proximidad para hacer clasificaciones o predicciones sobre la agrupación de un punto de datos individual



El algoritmo de búsqueda del vecino más próximo (KNN) en un KD-tree consiste en dos ideas clave: bajar por el árbol y luego retroceder podando ramas.

1) Descenso (como búsqueda binaria)

Dado un punto consulta q :

- Se empieza en la raíz
- En cada nodo se compara q con el punto del nodo en la dimensión de corte
- Se baja al hijo “más probable” (izquierda o derecha)

2) Inicializar mejor candidato

- se calcula la distancia al punto almacenado
- ese punto se toma como **mejor candidato actual**

3) Backtracking (subir el árbol)

Se vuelve hacia arriba en el árbol:

- en cada nodo se comprueba si el punto del nodo mejora la mejor distancia

4) Poda de ramas (la clave)

En cada nodo se decide si explorar el otro subárbol:

- se calcula la distancia de q al plano de división
- si esa distancia es **menor que la mejor distancia actual**,
👉 podría haber un punto más cercano → se explora la otra rama
- si no,
👉 se descarta completamente esa rama

Idea esencial

No se visitan todos los puntos

Se descartan regiones completas del espacio

Resumen del proceso

El algoritmo baja por el árbol hasta una hoja y luego retrocede explorando solo las ramas que podrían contener puntos más cercanos, podando el resto.